Ca' Foscari University of Venice

Development Methodology

Matteo Zanioli

matteo.zanioli@unive.it

Build vs Buy

- Organizations face a critical decision when integrating systems: **build** a custom solution tailored to their needs or **buy** an existing product from the market. This decision impacts cost, scalability, time-to-market, and long-term sustainability.

Pros & Cons

Aspect	Build (custom solution)	Buy (off-the-shelf solution)
Advantages	Tailored to specific business needs.	Faster implementation.
	Greater control over features and updates.	Lower upfront cost compared to custom development.
	Better Integration with existing systems.	Proven reliability and vendor support.
Disadvantages	High initial cost and resource investment.	Limited customization options.
	Longer development time.	Dependency on vendor for updates and support.
	Requires in-house expertise for maintenance.	Potential challenges in integrating with legacy systems.

Cost and Time Analysis

Build

- Initial cost: higher due to development effort.
- Time to market: slower because of design, development and testing.
- Long-term costs: can be lower if the solution avoids licensing fees and scales efficiently.

Buy

- Initial cost: lower, typiccaly involving licensing fees or subscriptions.
- Time to market: faster as solutions are pre-build.
- Long-term costs: May increase with recurring license fees and limited flexibility to scale.

Scalability and Customization

Build

- Scalability depends on the initial architecture. A modular design can ensure flexibility.
- High level of customization, allowing alignment with evolving business needs.

Buy

- Scalability may be limited by vendor constraints.
- Customization options vary but are often limited without vendor intervention.

Decision Framework

When to build

- Unique business processes that cannot be addressed by off-the-shelf solutions.
- Integration with highly specific legacy systems.
- Long-term cost-efficiency outweighs the initial investment.

When to buy

- Standardized requirements shared across industries.
- Immediate need for a solution.
- Limited in-house technical expertise for maintenance.

Key Questions to Evaluate:

- What is the total cost of ownership (TCO) over 5-10 years?
- Can off-the-shelf software meet at least 80% of the requirements?
- How critical is the solution for competitive differentiation?

Building custom solutions

- Modular and Scalable Approach:
  - Design systems using a modular architecture to ensure individual components can evolve independently.
  - Example: Separating data processing, user interfaces, and API layers.
- Best Practices for Integration:
  - Use open standards (e.g., REST, JSON, GraphQL) for APIs.
  - Ensure compatibility with third-party systems through extensible data models. Adopt CI/CD pipelines for rapid deployment and updates.

Integrating SaaS Solutions

- Challenges:
  - Data Silos: SaaS applications like Salesforce or Microsoft 365 may create isolated data repositories.
  - Latency and Reliability: SaaS solutions depend on internet connectivity, impacting response times.
  - Security and Compliance: Transferring sensitive data to cloud services requires stringent security measures.
  - Integration Complexity: Bridging SaaS systems with on-premise software often requires middleware.
- Solutions:
  - Use Integration Platform as a Service (iPaaS) tools like MuleSoft or Dell Boomi for seamless connectivity.
  - Implement APIs and webhooks for real-time data synchronization.
  - Conduct rigorous security assessments, focusing on data encryption and access controls.

Conclusion

- The **Build vs. Buy decision** requires careful consideration of costs, scalability, and business needs. While off-the-shelf solutions offer quick wins for standardized requirements, building custom solutions provides long-term flexibility and alignment with unique processes. In either case, focusing on modularity, integration capabilities, and alignment with business goals ensures sustainable success.
- Note that each decision required specific team organization to work properly. (If you offer a service, you have to change in order to manage a product).

Organizational Differences

- When discussing system integration and application development, it is crucial to understand the organizational and team structure differences. Both approaches require specific roles and skills, but their focus and workflow can vary significantly.

System Integration Team

- A system integration team focuses on combining and coordinating multiple software systems to function as a cohesive whole. The team often works with diverse technologies, platforms, and stakeholders.
- Key Focus:
  - Managing dependencies between systems.
  - Ensuring compatibility and consistent data flow.
  - Maintaining legacy systems while incorporating modern solutions.
- Composition:
  - Project Manager: Coordinates multiple vendors and teams, manages timelines, and ensures compliance with integration goals.
  - Solution Architect: Designs the integration blueprint, selecting middleware, protocols, and integration standards.
  - Integration Developers: Implement connections between systems, ensuring data flow and interoperability.
  - QA/Testers: Validate the correctness and reliability of integrations, including edge cases and performance tests.
  - Data Analysts: Handle data mapping, transformation, and integrity validation.

Ca' Foscari University of Venice

Application Development Team

- An application development team focuses on creating new software tailored to solve specific business problems or provide value to users. The work is typically confined to a single product or solution.
- Key Focus:
  - Delivering a standalone product.
  - Ensuring usability, functionality, and user experience.
  - Agile or DevOps-driven workflows to accelerate delivery.
- Composition:
  - Product Owner/Project Manager: Gathers requirements, prioritizes features, and oversees the development lifecycle.
  - Application Architect: Designs the overall application structure, ensuring scalability and performance.
  - Frontend and Backend Developers: Build the user interface and server-side logic.
  - QA/Testers: Conduct functional, usability, and performance testing.

Architect skills

System Integration

- Experience with middleware (e.g., ESB, APIs).
- Deep knowledge of protocols like SOAP, REST, AMQP, and standards like EDI.
- Expertise in mapping and transforming data across heterogeneous systems.

Application Development

- Proficiency in modern frameworks (e.g., React, Spring).
- Focus on scalability, maintainability, and software design patterns.
- Cloud-native architecture skills, including microservices and containerization.

Developer skills

System Integration

- Knowledge of scripting languages and middleware tools (e.g., MuleSoft, Apache Camel).
- Proficiency in data transformation languages (e.g., XSLT, JSON mapping).
- Familiarity with message brokers (e.g., Kafka, RabbitMQ).

Application Development

- Expertise in specific programming languages (e.g., Java, Python, JavaScript).
- Frontend and backend development skills.
- Knowledge of CI/CD pipelines and testing frameworks.

Key Challenges

System Integration

- Handling diverse systems and technologies.
- Maintaining system stability while implementing new integrations.
- Dependency on third-party vendors.

Application Development

- Pressure to innovate and meet market demands.
- Balancing technical debt with new feature development.
- Maintaining focus on user experience and satisfaction.

Use case: game development

- Scenario: Multiplayer Online Game (MMO)
- An international company is developing a massive multiplayer online (MMO) game that needs to scale to support millions of players in real time, operate across multiple platforms (PC, console, mobile), and be available globally.

Use case: application architecture

- Client-Server Architecture:
  - Client: The game on the user's device (PC, console, mobile).
    - Responsible for rendering graphics and user interaction.
  - Server: The backend that manages the game, synchronizes data between players, and stores persistent states.

- Microservices for the Backend: The backend architecture is based on microservices, each specialized for a specific function:
  - Matchmaking Service: Manages assigning players to game servers.
  - Game State Service: Maintains the real-time state of the game.
  - Player Profile Service: Handles persistent data such as statistics, inventories, and progress.
  - Analytics Service: Collects and analyzes player behavior data.
  - Authentication Service: Authenticates players via OAuth2 or integrations with providers like Google, Xbox, or Steam.

Use case: application architecture

- Database Layer:
  - NoSQL Databases (e.g., MongoDB, DynamoDB): For scalable data like player profiles and inventories.
  - SQL Databases (e.g., PostgreSQL): For relational data like financial transactions or global configurations.
  - In-Memory Databases (e.g., Redis): For high-speed temporary data such as game state.
- Scalability and Distribution:
  - Use of containers (e.g., Docker) and orchestrators like Kubernetes to scale microservices.
  - Content Delivery Network (CDN): For global distribution of assets like textures and updates.
  - Edge Computing: Reduces latency by moving some operations closer to players.

Use case: Integration Choice

- Real-Time Communication:
  - WebSocket Protocol: For real-time updates between client and server.
  - gRPC: Used between microservices for high-performance communication.
- Third-Party Integrations:
  - Game Engine: Unreal Engine or Unity, integrated with graphical and physics assets.
  - Cloud Services:
    - AWS GameLift: For provisioning and balancing game servers.
    - Google Cloud Spanner: For globally distributed databases.
    - Payments and Monetization: Integrations with systems like Stripe or PayPal for in-app purchases.
- Event-Driven Architecture:
  - Use of a Message Broker (e.g., Kafka or RabbitMQ) to handle events such as push notifications, daily rewards, or leaderboard updates.

Use case: challenges and integration solutions

- Data Synchronization Between Players:
  - Challenge: Ensuring game state consistency for all participants in real-time.
  - Solution:
    - Authoritative Server: A central server determines the game state, preventing inconsistencies caused by client latency.
    - Delta Updates: Only changes in the state are sent rather than the entire state.
- Managing Global Latency:
  - Challenge: Reducing latency for players across different continents.
  - Solution:
    - Regional servers connected via a global network.
    - Routing algorithms that assign players to the nearest server.
- Cross-Platform Compatibility:
  - Challenge: Enabling players on different platforms to play together.
  - Solution:
    - Standardized APIs to handle common interactions.
    - Normalization of inputs (e.g., controller vs. keyboard/mouse).
- Compliance and Security:
  - Challenge: Protecting player data and complying with regulations like GDPR.
  - Solution:
    - Encryption: Encrypt data both in transit and at rest.
    - Audit Trail: Monitor changes to personal data to ensure regulatory compliance.

- Fortnite's Architecture:
  - Microservices-Based Backend:
    - Uses Kubernetes to scale services such as matchmaking and account management.
  - SaaS Integration:
    - OAuth2 support for logins with Google, PlayStation Network, and Xbox Live accounts.
    - SaaS payment systems to handle microtransactions.
  - Cross-Platform Play:
    - Players on consoles, PCs, and mobile devices can play together.
    - Cloud synchronization for progress and purchases.
- Results:
  - Millions of simultaneous players.
  - Service continuity even during global updates, thanks to distributed systems.
  - Effective monetization with personalized in-game purchases.

Ca' Foscari University of Venice

FORTNITE

- Concurrent Users (peak): 15.3M+ (during special events).
- Concurrent Users (daily average): Millions of wolrdwide players.
- Monthly Active Users (MAU): 350M registered users.
- Client-server calls per second:
  - Matchmaking and authentication: 100.000+ of requests per second during peak
  - WebSocket and Real-Time updates: 30.000+ messages per second per game server.
- Latency: 20-50ms for most players using regionally distributed servers.
- Game severs in operation: 1000+ of server instance on cloud providers.
  - Each server suppors 100-200 players per match
- DB queries for player profiles: 2M+ of queries per day.
- Logged Events: 2TB+/daily for every player.
- CDN: handling 2PB+ of monthly traffic.
- Cost per player: about $0.01 per player per day on cloud infrastructure, depending on global demand.

Conclusion

- Developing global-scale video games requires a modular, scalable, and secure architecture. The choice of technologies, methodologies, and integrations depends on specific needs, but combining microservices, real-time communication, and cross-platform standards is essential for providing an optimal player experience.